



Stable-Baselines3

Status

- Week 1: Reinforcement Learning Basics
- Week 2: Basic Q-Learning with Basic "homemade" environment class
- Week 3:
 - Gymnasium custom environment: an upgrade for our environment class
 - Pygame for rendering
 - BlocksWorld-v0 environment
- Week 4:
 - Stable-Baselines3

Observation and Action Spaces

- <https://gymnasium.farama.org/api/spaces/fundamental/#>

- Box Example:

```
obs_space = Box(low=-1.0, high=2.0, shape=(3, 4), dtype=np.float32)
```

- Box for 100x130 pixel images with RGB pixels

```
obs_space2 = Box(low=0, high=255, shape=(100, 130, 3), dtype=np.uint8)
```

```
obs_space.sample()
```

```
action_space = Discrete(2, seed=42) # action = 0 or 1
```

```
action_space.sample()
```

Stable-Baselines 3

Base RL Class

<https://stable-baselines3.readthedocs.io/en/master/modules/base.html>

- Common interface for all the RL algorithms (which extend the Base Class)

```
model = DQN("MultiInputPolicy", env, verbose=1)
```

- Learn method of these classes

```
model.learn(total_timesteps=10000, log_interval=4)
```

Which Algorithm?

https://stable-baselines3.readthedocs.io/en/master/guide/rl_tips.html#which-algorithm-should-i-use

Stable-Baselines3 features

- (Hybrids) Video at https://www.youtube.com/watch?v=lknegt0_DXJg
 - Advice/tips on RL, and Stable-Baselines3 hands on tutorial
- Tutorial at <https://github.com/araffin/rl-handson-rlvs21>
- Jupyter exercise: https://colab.research.google.com/github/araffin/rl-handson-rlvs21/blob/main/rlvs_hands_on_sb3.ipynb

Stable-Baselines3 features

- Vectorized Environments

Vectorized Environments are a method for stacking multiple independent environments into a single environment.

Instead of training an RL agent on 1 environment per step, it allows us to train it on n environments per step.

- For our examples, grid_worlds, blocks worlds, etc, running on our VM and/or loaner laptop, we use a simple environment which SB3 will turn into a DummyVecEnv for compatibility with VecEnv

Stable-Baselines3 features

- Vectorized Environments example

```
vec_env = make_vec_env("Pendulum-v1", n_envs=16)

# At each call to `env.step()`, 16 transitions will be collected, so we account for that for fair comparison

model = PPO("MlpPolicy", vec_env, n_epochs=1, n_steps=n_steps // 16, verbose=1).learn(int(2e4))
```

- Detailed example:

https://colab.research.google.com/github/Stable-Baselines-Team/rl-colab-notebooks/blob/sb3/multiprocessing_rl.ipynb

Stable-Baselines3 features

- Callbacks
 - A callback is a set of functions that will be called at given stages of the training procedure. You can use callbacks to access internal state of the RL model during training. It allows one to do monitoring, auto saving, model manipulation, progress bars, ...
 - CheckpointCallback and EvalCallback included in SB3

Stable-Baselines3 features (using EvalCallback)

```
import gymnasium as gym

from stable_baselines3 import SAC

from stable_baselines3.common.callbacks import EvalCallback

# Separate evaluation env
eval_env = gym.make("Pendulum-v1")

# Use deterministic actions for evaluation
eval_callback = EvalCallback(eval_env, best_model_save_path="./logs/",
                             log_path="./logs/", eval_freq=500,
                             deterministic=True, render=False)

model = SAC("MlpPolicy", "Pendulum-v1")
model.learn(5000, callback=eval_callback)
```

HyperParameter Tuning

From https://colab.research.google.com/github/araffin/rl-handson-rlvs21/blob/main/rlvs_hands_on_sb3.ipynb#scrollTo=uDUfeZcyjPKS

```
import torch.nn as nn
policy_kwargs = dict(
    net_arch=[
        dict(vf=[64, 64], pi=[64, 64]), # network architectures for actor/critic
    ],
    ortho_init=True, # Orthogonal initialization,
    activation_fn=nn.Tanh,
)
hyperparams = dict(
    n_steps=5,
    learning_rate=7e-4,
    gamma=0.99, # discount factor
    gae_lambda=1.0, # Factor for trade-off of bias vs variance for Generalized Advantage Estimator
                  # Equivalent to classic advantage when set to 1.
    max_grad_norm=0.5, # The maximum value for the gradient clipping
    ent_coef=0.0, # Entropy coefficient for the loss calculation
)

model = A2C("MlpPolicy", "CartPole-v1", seed=8, verbose=1, **hyperparams).learn(budget)
```